VINGO

Enterprise Food Delivery Ecosystem

Complete Software Requirements Specification · System Design · Architecture Blueprint · Execution Roadmap

Document Version	2.0 — Production-Grade
Classification	Resume Flagship Project · Public Portfolio
Stack	MERN · Socket.IO · Redis · Docker · GitHub Actions
Coverage	Auth · Orders · Payments · Real-time GPS · AI · Analytics · DevOps
Target Role	Full-Stack / Backend / Software Engineer (SDE I / II)
Inspired By	Swiggy · Zomato · UberEats — production architecture

KEY PERFORMANCE TARGETS

>98%	<300ms	>90	<5 min
Order Success Rate	P95 API Latency	Lighthouse Score	ETA Prediction Error

TECHNOLOGY STACK

React 18 · Node.js 20 · Express · MongoDB · Redis · Socket.IO · Docker · GitHub Actions

Razorpay · JWT · Bcrypt · Helmet · Mongoose · Sentry · Grafana · k6

Cohere AI · Cloudinary · Nginx · PM2 · Jest · Cypress · MapboxGL · PWA

This document provides a production-grade specification for Vingo — a full-stack, real-time food delivery platform built on the MERN stack. It covers every layer from UX flows to database schemas, API contracts, socket events, security hardening, DevOps pipelines, and AI-powered features. The depth and breadth of this specification demonstrate senior-engineer thinking and system design competence to technical recruiters.

TABLE OF CONTENTS

1 PRODUCT VISION & GOALS

Building a production-grade, recruiter-ready food delivery ecosystem

Vingo is a full-scale food delivery platform modelled after Swiggy, Zomato, and UberEats. It serves four distinct actor groups — Consumers, Restaurant Owners, Delivery Partners, and Super Admins — through three dedicated front-ends and a shared microservice-oriented Node.js backend. Every design decision prioritises observability, security, and horizontal scalability from day one.

Core Product Pillars

- **Reliability** — Order success rate >98 % with idempotent payment processing, dead-letter queues, and automatic retries.
- **Performance** — P95 API latency under 300 ms enforced via Redis caching, database indexing, and CDN-served static assets.
- **Real-time** — Sub-second GPS updates and order status pushes over Socket.IO with room-based fan-out and Redis pub/sub for multi-node support.
- **Security** — JWT + refresh token rotation, bcrypt, Helmet, rate-limiting, RBAC middleware, and CSRF protection on every mutating request.
- **Intelligence** — AI-powered meal recommendations, dynamic surge pricing, predictive ETA, and fraud-detection heuristics.
- **Developer Experience** — Dockerised development, GitHub Actions CI/CD, Sentry error tracking, and Grafana dashboards from the first commit.

Success Metrics

>98%	<300ms	>90	<5 min
Order Success Rate	P95 API Latency	Lighthouse Score (Mobile)	ETA Prediction Error

Non-Functional Requirements

Availability	99.9% uptime SLA; blue-green deployments; DB replica failover <30 s
Throughput	10,000 concurrent WebSocket connections per node; horizontal pod auto-scaling
Data Retention	Orders retained 7 years for auditing; GPS pings TTL 72 h; logs 30 days
Compliance	PCI-DSS compliant payment handling; GDPR-ready data export & deletion APIs
Accessibility	WCAG 2.1 AA; keyboard navigation; screen-reader ARIA labels
Internationalisation	i18n via react-intl; EN / HI / TA / KN locale bundles at launch

2 ROLES, ACTORS & RBAC MATRIX

Granular permission model enforced at both middleware and resolver level

Vingo uses a flat Role-Based Access Control model with four roles stored in the JWT payload and validated by a dedicated `requireRole(...roles)` Express middleware on every protected route. Row-level security additionally prevents cross-tenant data access for restaurant owners.

Feature	User	Restaurant Owner	Delivery Partner	Super Admin
Browse Restaurants & Menus	✓	✓	✓	✓
Place / Cancel Orders	✓	✗	✗	✓
Track Live Delivery	✓	✓	✓	✓
Manage Restaurant & Menu	✗	✓	✗	✓
Accept / Reject Orders	✗	✓	✗	✓
View Own Revenue Analytics	✗	✓	✗	✓
Accept Delivery Assignment	✗	✗	✓	✓
Update GPS / Location	✗	✗	✓	✓
View Delivery History & Pay	✗	✗	✓	✓
Manage All Users & Restaurants	✗	✗	✗	✓
Platform-Wide Analytics	✗	✗	✗	✓
Issue Refunds & Coupons	✗	✗	✗	✓
Fraud Detection Controls	✗	✗	✗	✓

RBAC Middleware Implementation

The `requireRole` middleware extracts the role claim from the verified JWT, checks against the allowed roles array, and returns `403 Forbidden` with a structured error object on mismatch. Restaurant ownership is verified by comparing `req.user._id` against the restaurant's `ownerId` field before any mutation.

3 FEATURE SPECIFICATION

Complete acceptance criteria for every module

Authentication & Identity

- Email/password signup with Zod validation; bcrypt(12) password hashing
- Google OAuth 2.0 via Passport.js; auto-account-link on email match
- Stateless JWT (access 15 min) + HttpOnly refresh token (7 days) stored in cookie
- Refresh token rotation: old token invalidated on every refresh
- Password reset via time-limited (15 min) HMAC token sent to email (Nodemailer + SES)
- Account lockout after 5 failed attempts; TOTP 2FA for Admin/Owner roles
- Session audit log: device fingerprint, IP, user-agent, timestamp stored in AuditLogs

Restaurant & Menu Discovery

- Full-text search across restaurant name, cuisine, and dish with MongoDB Atlas Search
- Multi-facet filtering: cuisine, rating ≥ N, price band, dietary (veg/vegan/jain), delivery time
- Geo-proximity sort using MongoDB 2dsphere index; configurable radius (default 5 km)
- Menu versioned on publish; previous version retained for in-flight order accuracy
- Availability windows per item (breakfast/lunch/dinner); auto-hide outside window
- Restaurant profile: cover photo, gallery (S3), FSSAI cert, avg prep time, rating breakdown
- Trending tags computed nightly via aggregation pipeline on order frequency

Cart & Checkout

- Redis-backed cart with TTL 2 hours; cart migrated from guest → authenticated session
- Cross-device cart sync via userId key in Redis; merge strategy on conflict
- Customisation options (size, spice level, add-ons) stored per cart item
- Real-time item availability check on cart open and on checkout attempt
- Coupon engine: percentage-off, flat-off, BXGY, minimum order, single-use, user-specific
- Loyalty points preview before order: earn X points + wallet cashback shown
- Delivery fee matrix: base + per-km rate; surge multiplier from Delivery Intelligence Engine
- GST breakdown (CGST + SGST) computed server-side; never trusted from client

Order Lifecycle & State Machine

- States: PLACED → PAYMENT_PENDING → PAYMENT_CONFIRMED → ACCEPTED → PREPARING → READY → PICKED_UP → OUT_FOR_DELIVERY → DELIVERED | CANCELLED | REFUND_INITIATED
- Each transition timestamped and appended to orders.statusTimeline array (append-only)
- Restaurant can reject with reason; auto-refund triggered within 5 minutes via queue
- Cancellation window: 60 s after placement for free cancellation; partial refund after
- Scheduled orders: cron-triggered PLACE event up to 24 hours in advance
- Bulk order support for corporate accounts with split-billing across cost centres
- Email + push notification on every state transition (FCM + Expo Push via worker queue)

Payments & Wallet

- Razorpay integration: orders API → payment capture → webhook verification (HMAC-SHA256)
- Idempotency key per order prevents double-charge on retry
- In-app wallet: top-up (Razorpay), auto-debit on order, credit on refund/cashback
- UPI, Net Banking, Cards, Wallets (Paytm/PhonePe) supported via Razorpay methods
- COD supported with max-order-value guard and fraud-score threshold
- Refund engine: instant to wallet; 5–7 days to original payment method via Razorpay Refund API
- Wallet transaction ledger with debit/credit type, reference, balance-after on every row

Real-time GPS Tracking

- Delivery partner emits gps_ping every 3 seconds via Socket.IO (lat, lng, heading, speed)
- Smoothed polyline rendered on customer map using Kalman filter approximation
- ETA recalculated every 90 s using Mapbox Directions API + historical speed data
- OTP delivery confirmation: 6-digit code generated server-side, pushed to delivery partner when within 50 m
- Customer can share live tracking link (JWT-signed, 4-hour expiry) with non-app users
- Breadcrumb trail stored in DeliveryTracking collection; visualised on Admin heatmap

Reviews & Ratings

- Post-delivery review prompt triggered 15 min after DELIVERED status via FCM
- Dual rating: Food (1–5 stars) + Delivery experience (1–5 stars) stored separately
- Photo upload with review (max 3 images, Cloudinary CDN)
- Restaurant reply to reviews with audit trail
- Abuse detection: ML keyword filter + manual moderation queue for flagged reviews
- Rolling average recomputed via MongoDB aggregation pipeline on each new review
- Review analytics in Owner Dashboard: trend over time, keyword cloud

Loyalty & Gamification

- Earn 1 point per ■10 spent; double points on partner restaurants on weekends
- Tier system: Bronze → Silver → Gold → Platinum with spend thresholds
- Tier benefits: free delivery, priority support, exclusive deals, higher cashback %
- Referral system: ■50 credit to referrer + ■75 to referee after referee's first order
- Streak rewards: order 7 days in a row → bonus 200 points + surprise coupon
- Points expiry: 12-month inactivity window; email warning 30 days before expiry
- Leaderboard (opt-in) for power users in a city; Top 10 monthly reward

Notifications

- In-app, Email, SMS (Twilio), and Push (FCM) channels per event
- User-configurable notification preferences per channel per event type
- BullMQ worker queue for reliable async delivery with exponential backoff retry
- Transactional email templates via React Email + AWS SES; open/click tracking
- Order confirmation, delivery updates, payment receipts, review prompts, promotions
- Admin broadcast: push notification campaigns with audience segmentation

4 UI/UX SCREEN CATALOGUE

All 22 screens across 3 apps — User, Owner, Admin

User App

Landing / Home	Hero banner with city selector, trending restaurants carousel, cuisine filter chips, promotional banners (A/B tested), search bar with voice input
Search Results	Infinite scroll list with skeleton loaders; live filter panel; sort by: relevance / rating / delivery time / price; map toggle view
Restaurant Page	Cover + gallery; rating breakdown; offer badges; menu with sections, item cards (photo, price, veg/non-veg indicator, customisation accordion); sticky Add to Cart CTA
Cart	Item list with quantity controls; coupon field with instant validation; price breakdown (subtotal, discount, delivery, taxes, total); wallet toggle; estimated delivery time
Checkout	Address selection / new address via Google Places Autocomplete; payment method picker; schedule delivery option; order summary confirmation
Order Tracking	Live map with delivery partner pin + polyline; ETA chip; order status timeline (stepper); chat button; cancel window countdown
Order History	Paginated list with status badge; re-order deep-link; download invoice PDF; review CTA for completed orders
Profile & Settings	Edit info; address book; payment methods; notification prefs; loyalty tier card; referral code; delete account (GDPR)
Wallet	Balance card; top-up flow; transaction ledger with type chips and search

Restaurant Owner App

Owner Dashboard	Today's revenue KPIs; order count by status; peak-hour bar chart; avg prep time; top dishes; low-stock alerts
Order Queue	Kanban columns: New / Preparing / Ready; accept/reject with reason; estimated prep time input; print order ticket (thermal printer API)
Menu Manager	Category + item CRUD; drag-to-reorder; bulk availability toggle; image upload; scheduled publish; CSV import
Analytics	Revenue over time (daily/weekly/monthly); item-level performance; customer retention cohort; heatmap of order origins; export to CSV
Reviews	Star breakdown; reply to reviews; flag inappropriate content; keyword sentiment trend

Super Admin Panel

Global Dashboard	Platform GMV; new users/restaurants/orders today; active delivery partners; real-time order map (Mapbox)
User Management	Search/filter all users; impersonate; suspend; view order + wallet + audit history; manual wallet credit
Restaurant Management	Approve/reject onboarding; toggle availability; view revenue; override menu; compliance docs
Delivery Partner Management	Approve documents; assign zones; view GPS history; rating & completion rate; earnings reconciliation

Coupon & Campaign Studio	Visual coupon builder; audience segmentation (geo, tier, order count); push-notification campaign scheduler
Fraud & Risk Console	Real-time fraud alert feed; manual review queue; IP block list; chargeback management
Platform Analytics	Cohort retention; LTV model; city-wise GMV heatmap; delivery SLA breach report; refund rate trend

5 SYSTEM ARCHITECTURE

Layered, service-oriented architecture with event-driven real-time layer

Vingo uses a layered architecture: a React SPA (Vite) communicates through an Nginx reverse-proxy / API Gateway to a Node.js + Express monolith organised into domain modules (auth, restaurants, orders, tracking, payments, notifications, admin). Redis serves as both a cache layer and a pub/sub bus for Socket.IO horizontal scaling. MongoDB Atlas (M10+) handles persistent storage with replica sets for high availability. Async workloads are handled by BullMQ workers connected to a Redis queue.

Client Layer	React 18 SPA (Vite) · React Native (Expo) for mobile · PWA manifest + service worker · Socket.IO client v4 · Mapbox GL JS for live tracking
CDN / Edge	Cloudflare CDN for static assets · Image optimisation (Cloudinary) · DDoS protection · Edge caching headers (Cache-Control, ETag)
API Gateway / Reverse Proxy	Nginx: SSL termination · Rate limiting (limit_req_zone) · Load balancing to N Node.js instances · WebSocket upgrade (proxy_pass)
Application Layer	Node.js 20 LTS + Express 5 · Domain modules: auth, restaurants, orders, payments, tracking, notifications, coupons, reviews, admin · BullMQ workers for email, SMS, push, refunds
Cache Layer	Redis 7 (AWS ElastiCache): session store · cart (TTL 2 h) · rate-limit counters · pub/sub for Socket.IO · AI recommendation cache (TTL 1 h)
Database Layer	MongoDB Atlas M10 (3-node replica set) · Collections: Users, Restaurants, Menus, Orders, DeliveryTracking, Notifications, Wallets, Reviews, Coupons, AuditLogs · Indexes: 2dsphere, text, TTL, compound
Real-time Layer	Socket.IO v4 with Redis adapter (socket.io-redis) · Rooms: order:{id}, restaurant:{id}, admin:live · Events: gps_ping, order_update, eta_update, otp_request
Monitoring & Observability	Sentry (error tracking + performance) · Grafana + Prometheus (metrics) · Winston + Morgan (structured JSON logs → CloudWatch) · Uptime Robot (external pings) · PagerDuty (on-call alerts)

Request Flow — Order Placement

- 1 · User taps Place Order on React SPA
- 2 · POST /api/orders hits Nginx → Node.js; requireRole('user') middleware validates JWT
- 3 · Cart fetched from Redis; idempotency key checked (prevent duplicate orders)
- 4 · Razorpay Order created; orderId + amount returned to client
- 5 · Client completes payment; Razorpay redirects with paymentId + signature
- 6 · POST /api/payments/verify: HMAC-SHA256 signature verified server-side
- 7 · Order written to MongoDB (status: PAYMENT_CONFIRMED); wallet/points updated
- 8 · BullMQ job enqueued: send confirmation email + push notification
- 9 · Socket.IO emits order_placed to room restaurant:{restaurantId}
- 10 · Restaurant owner's dashboard updates in real-time; timer starts for auto-accept (5 min)
- 11 · On acceptance: Delivery Intelligence Engine scores + assigns nearest partner
- 12 · GPS pings start; ETA computed; Socket.IO pushes to room order:{orderId}
- 13 · On delivery: OTP verified; status → DELIVERED; review prompt scheduled (+15 min)

6 DATABASE SCHEMA DESIGN

MongoDB collections with full field specs, indexes and relationships

Users Collection

Field	Type	Constraints	Description
_id	ObjectId	PK, auto	MongoDB document ID
role	String	enum, required	'user' | 'owner' | 'delivery' | 'admin'
name	String	required, trim	Full display name
email	String	unique, lowercase	Verified email address
passwordHash	String	bcrypt(12)	Null for OAuth-only accounts
googleId	String	sparse index	OAuth sub claim
phone	String	E.164 format	For SMS notifications
addresses	[AddressSchema]	embedded array	Saved delivery addresses with label, coords
walletBalance	Number	default 0, min 0	Paise (integer); never Float
loyaltyPoints	Number	default 0	Earned points balance
loyaltyTier	String	enum	'bronze'|'silver'|'gold'|'platinum'
referralCode	String	unique	6-char alphanumeric; auto-generated on signup
isActive	Boolean	default true	False = soft-deleted or suspended
lastLogin	Date		Updated on every successful auth
createdAt	Date	auto	TTL index candidate for GDPR right-to-erasure

Restaurants Collection

Field	Type	Constraints	Description
_id	ObjectId	PK	
ownerId	ObjectId	ref: Users	Restaurant owner; used for row-level auth
name	String	required, text-indexed	Full-text search field
cuisines	[String]	enum array	Cuisine tags for filtering
location	GeoJSON Point	2dsphere index	{ type:'Point', coordinates:[lng,lat] }
address	AddressSchema	embedded	Street, city, pincode, landmark
coverImage	String	Cloudinary URL	CDN-served cover photo
avgRating	Number	0–5, 1 d.p.	Recomputed on each new review
totalReviews	Number		Denormalised count
isOpen	Boolean		Real-time toggle by owner; cron resets by schedule
openingHours	[DaySchedule]	embedded	Mon–Sun open/close times
avgDeliveryMin	Number		ETA baseline for listing card
priceForTwo	Number		Used for price-band filter (■/■■/■■■)
isApproved	Boolean	admin only	Platform onboarding gate
isDeleted	Boolean	soft delete	Preserves historical order references

Orders Collection

Field	Type	Constraints	Description
_id	ObjectId	PK	
userId	ObjectId	ref: Users, indexed	
restaurantId	ObjectId	ref: Restaurants, indexed	
deliveryPartnerId	ObjectId	ref: Users, nullable	Assigned after acceptance
items	[OrderItem]	embedded	snapshot of item name/price at order time
status	String	enum, indexed	Full state machine string
statusTimeline	[{status,ts}]	append-only	Audit trail of all transitions
subtotal	Number	paise	Pre-discount item total
discountAmount	Number	paise	Coupon / offer deduction
deliveryFee	Number	paise	Computed by Delivery Engine
taxAmount	Number	paise	CGST+SGST server-computed
totalAmount	Number	paise	Final charged amount
paymentMethod	String	enum	'razorpay'|'wallet'|'cod'
razorpayOrderId	String		For webhook reconciliation
couponCode	String	nullable	Applied coupon reference
deliveryAddress	AddressSchema	snapshot	Address at time of order
otp	String	6-digit hash	For delivery confirmation
isScheduled	Boolean		Scheduled order flag
scheduledFor	Date	nullable	Target delivery time
estimatedETA	Date		Updated by ETA engine
createdAt	Date	TTL candidate	Order creation timestamp

DeliveryTracking Collection

Field	Type	Constraints	Description
`_id`	`ObjectId`	PK	
`orderId`	`ObjectId`	ref: Orders, indexed	
`partnerId`	`ObjectId`	ref: Users	
`location`	`GeoJSON Point`	2dsphere	Live coordinate
`heading`	`Number`	0–359°	Compass bearing for map arrow
`speed`	`Number`	km/h	From device GPS
`accuracy`	`Number`	metres	GPS accuracy radius
`timestamp`	`Date`	TTL: 72 h	Auto-deleted after 72 hours

Additional Collections Summary

Menus	restaurantId, categories[], items[] with variants/extras; versioned on publish
Wallets	userId (unique), balance, transactions[] with type/amount/ref/balanceAfter
Reviews	orderId, userId, restaurantId, foodRating, deliveryRating, photos[], reply
Coupons	code (unique), type, value, minOrder, maxUses, perUserLimit, expiry, audience
Notifications	userId, channel, type, payload, isRead, sentAt, readAt
AuditLogs	actorId, actorRole, action, targetId, targetModel, diff, ip, userAgent, ts

7 REST API REFERENCE

Complete endpoint catalogue with auth requirements and status codes

Authentication Endpoints

Method	Endpoint	Auth	Description	Status Codes
POST	/api/auth/register	Public	Create new user account with email + password	201, 400, 409
POST	/api/auth/login	Public	Returns access JWT + sets refresh cookie	200, 401, 423
POST	/api/auth/refresh	Cookie	Rotate refresh token; return new access JWT	200, 401
POST	/api/auth/logout	JWT	Invalidate refresh token in DB	204, 401
POST	/api/auth/google	Public	OAuth code exchange; returns JWT pair	200, 400
POST	/api/auth/forgot-password	Public	Send HMAC reset link to email	202, 404
POST	/api/auth/reset-password	Token	Validate token; update password hash	200, 400, 410
GET	/api/auth/me	JWT	Return authenticated user profile	200, 401

Restaurant Endpoints

Method	Endpoint	Auth	Description	Status Codes
GET	/api/restaurants	Public	List with geo-filter, cuisine, rating, price-band query params	200
GET	/api/restaurants/:id	Public	Full restaurant profile + active menu	200, 404
POST	/api/restaurants	Owner	Create restaurant; returns pending-approval status	201, 400, 403
PUT	/api/restaurants/:id	Owner	Update profile fields; ownership check	200, 403, 404
PATCH	/api/restaurants/:id/status	Owner	Toggle isOpen; respects opening hours	200, 403
DELETE	/api/restaurants/:id	Admin	Soft-delete; orphaned orders preserved	204, 403
GET	/api/restaurants/:id/analytics	Owner	Revenue, orders, ratings over date range	200, 403

Menu Endpoints

Method	Endpoint	Auth	Description	Status Codes
GET	/api/restaurants/:id/menu	Public	Active menu with categories and items	200, 404
POST	/api/restaurants/:id/menu/items	Owner	Add item with image upload (multipart)	201, 400, 403
PUT	/api/menu/items/:itemId	Owner	Update item details or price	200, 403, 404
PATCH	/api/menu/items/:itemId/toggle	Owner	Toggle item availability	200, 403
DELETE	/api/menu/items/:itemId	Owner	Soft-delete item; hide from menu	204, 403
POST	/api/restaurants/:id/menu/publish	Owner	Snapshot and publish menu version	200, 403

Order Endpoints

Method	Endpoint	Auth	Description	Status Codes
POST	/api/orders	User	Place order; validates cart, applies coupon, creates Razorpay order	201, 400, 402
GET	/api/orders	User	Paginated order history with status filter	200, 401
GET	/api/orders/:id	User/Owner/Delivery	Full order detail	200, 403, 404
PATCH	/api/orders/:id/cancel	User	Cancel within window; trigger refund	200, 400, 409
PATCH	/api/orders/:id/accept	Owner	Accept with prep time; triggers delivery assignment	200, 403
PATCH	/api/orders/:id/reject	Owner	Reject with reason; auto-refund enqueued	200, 403
PATCH	/api/orders/:id/ready	Owner	Mark food ready; delivery partner notified	200, 403
PATCH	/api/orders/:id/pickup	Delivery	Confirm pickup; status → OUT_FOR_DELIVERY	200, 403
POST	/api/orders/:id/verify-otp	Delivery	Submit OTP to complete delivery	200, 400, 410
GET	/api/orders/:id/invoice	User	Download PDF invoice (Puppeteer-rendered)	200, 403, 404

Payment & Wallet Endpoints

Method	Endpoint	Auth	Description	Status Codes
POST	/api/payments/verify	User	Verify Razorpay signature; confirm payment	200, 400, 402
POST	/api/payments/webhook	Razorpay	Async event handler (HMAC-verified)	200, 400
GET	/api/wallet	JWT	Balance + paginated transaction ledger	200, 401
POST	/api/wallet/topup	User	Create Razorpay order for wallet top-up	201, 400
POST	/api/wallet/topup/verify	User	Verify payment; credit wallet	200, 400
POST	/api/refunds	Admin	Manual refund trigger with reason	201, 400, 403

Tracking & Delivery Endpoints

Method	Endpoint	Auth	Description	Status Codes
GET	/api/tracking/:orderId	User/Owner	Latest delivery partner location + ETA	200, 403, 404
PUT	/api/delivery/location	Delivery	Batch GPS update (up to 5 pings)	200, 400, 401
GET	/api/delivery/active-order	Delivery	Current assigned order details	200, 401, 404
GET	/api/delivery/earnings	Delivery	Daily/weekly earnings breakdown	200, 401
POST	/api/tracking/:orderId/share-link	User	Generate shareable tracking JWT	201, 403

8 REAL-TIME EVENT SYSTEM (SOCKET.IO)

Full duplex WebSocket layer with Redis adapter for horizontal scaling

All Socket.IO connections require a valid JWT passed as `auth.token` in the handshake. The server validates the token in the `io.use()` middleware and attaches the decoded user to `socket.data.user`. Rooms are used for scoped fan-out: `order:{id}`, `restaurant:{id}`, and `admin:live`. The Redis adapter (`@socket.io/redis-adapter`) enables events to be broadcast across multiple Node.js instances.

Event Name	Direction	Audience	Payload / Behavior
`order_placed`	Client → Server	User	Triggered on checkout success; contains orderId, items, coords
`order_accepted`	Server → Client	Owner	Restaurant confirms; ETA computed; delivery pool notified
`order_rejected`	Server → Client	User	Payment reversed; reason code returned
`delivery_assigned`	Server → Client	User + Owner	DeliveryPartnerId, name, vehicle, photo, phone hash
`gps_ping`	Client → Server	Delivery	lat, lng, heading, speed, accuracy; stored in Redis + Mongo
`location_update`	Server → Client	User + Owner	Smoothed position for map polyline rendering
`eta_update`	Server → Client	User	Revised ETA based on traffic; updates every 90 s
`otp_request`	Server → Client	Delivery	6-digit OTP sent when within 50 m radius of customer
`delivery_completed`	Server → Client	User + Owner	Order status → DELIVERED; review prompt triggered
`order_update`	Server → Client	User	Generic status machine: PLACED→ACCEPTED→PREPARING→READY→PICKED→DELIVERED
`payment_webhook`	Server Internal	Admin	Razorpay async event; idempotency key checked before processing
`fraud_alert`	Server → Admin	Admin	Spike detected; auto-suspend + Slack notification

9 PAYMENT & WALLET ENGINE

Razorpay integration with idempotency, webhook processing and wallet ledger

Payment Flow

- Client calls POST /api/orders → server validates cart + coupon → creates Razorpay Order (server-side)
- Razorpay orderId + key returned to client; Razorpay Checkout SDK opens
- User completes payment; Razorpay calls client success callback with paymentId + orderId + signature
- Client calls POST /api/payments/verify; server recomputes HMAC-SHA256(orderId|paymentId) with secret
- If signature matches: payment captured; order status → PAYMENT_CONFIRMED; idempotency key stored in Redis (24 h TTL)
- BullMQ job enqueues: confirmation email, push notification, loyalty points award
- Razorpay also fires async webhook → POST /api/payments/webhook; server re-verifies and reconciles

Wallet Architecture

- Wallet balance stored in paise (integer) to avoid floating-point rounding errors
- All mutations use MongoDB transactions: debit wallet + write transaction record atomically
- Concurrent debit protected by optimistic lock: findOneAndUpdate with balance >= amount condition
- Cashback credited asynchronously via BullMQ worker after order DELIVERED
- Refund priority: wallet (instant) → original payment method (5–7 days via Razorpay Refunds API)
- Top-up via Razorpay; supports UPI, Net Banking, Cards; max ■10,000 per transaction

10 DELIVERY INTELLIGENCE ENGINE

Route optimisation, smart assignment, fraud detection and heatmaps

Delivery Partner Assignment Algorithm

- On order ACCEPTED: query MongoDB for delivery partners with status='available' within 3 km using $nearSphere; sorted by composite score
- Composite score: distance weight (40%) + current rating weight (30%) + acceptance rate weight (20%) + shift time remaining (10%)
- Assignment offer sent via Socket.IO; partner has 30 s to accept; auto-decline triggers next candidate (max 3 attempts before alert)
- Accepted assignments locked in Redis to prevent double-assignment race condition

ETA Prediction Engine

- Baseline ETA = restaurant avg prep time + (Mapbox Directions API route duration × traffic multiplier)
- Traffic multiplier fetched from Mapbox Traffic API; cached in Redis per corridor (TTL 5 min)
- ETA refined every 90 s using partner's last 5 GPS pings and current speed
- ML correction factor (XGBoost model, retrained weekly) adjusts for historical SLA breaches by area + time-of-day
- ETA pushed to customer via Socket.IO eta_update; accuracy tracked for model retraining

Surge Pricing Engine

- Demand/supply ratio computed per 1 km² grid cell every 2 minutes using Redis counters
- Surge multiplier (1.0×–2.5×) applied to delivery fee; not to food price
- Surge badge shown on restaurant cards and at checkout with explanation copy
- Admin can override or cap surge per city; changes effective within 60 s

Fraud Detection

- Velocity checks: >3 failed payment attempts in 10 min → temporary block
- Geo-mismatch: delivery address vs. IP geolocation distance > 500 km → manual review
- Abnormal order patterns: >10 orders/day per account → KYC verification flag
- COD fraud: partner marks delivered without OTP match → auto-investigation queue
- Chargeback spike: >2% rate for restaurant triggers payout hold + admin alert

11 AI & ML FEATURES

Intelligent recommendations, predictions and personalisation

AI Meal Recommendation Engine

Cohere Embed API converts user order history and item descriptions into semantic vectors stored in a pgvector (Supabase) table. On homepage load, the top-5 items with highest cosine similarity to the user's taste vector are retrieved and rendered as 'Picked For You' cards. Cache TTL: 1 hour per userId in Redis.

Predictive ETA (ML Model)

XGBoost regression model trained weekly on 90 days of order data. Features: restaurant prep time histogram, partner speed percentiles by zone+hour, day-of-week + hour encoding, weather proxy (rain flag via Open-Meteo API). Model served via a lightweight Flask endpoint; called by Node.js at order acceptance.

Dynamic Surge Pricing

Demand signal: open orders per grid cell. Supply signal: active delivery partners per cell. Multiplier formula: max(1.0, min(2.5, demand / supply × base_rate)). Computed every 2 min by a BullMQ cron worker; published to Redis; read by API on cart price computation.

Smart Search (Semantic + Keyword)

MongoDB Atlas Search (Lucene) for keyword search on restaurant name, cuisine, dish name. Cohere re-ranking re-orders top-20 keyword results by semantic relevance to query. Typo-tolerance via fuzzy matching; synonym expansion (e.g. 'biryani' → 'biriyani').

Churn Prediction

Logistic regression model trained on 30-day cohorts. Features: days-since-last-order, order frequency delta, avg rating given, coupon redemption rate. Users with churn probability >70% enter a 'win-back' campaign segment receiving a personalised coupon via scheduled notification.

Review Abuse Detection

Cohere Classify API fine-tuned on 2,000 labelled samples (legitimate vs abusive reviews). Reviews scoring >0.85 abuse confidence are auto-held in moderation queue. False-positive appeals handled via Admin console.

12 SECURITY ARCHITECTURE

Defence-in-depth: network, application, data and supply-chain layers

JWT Strategy	Access token: 15 min, RS256 signed; Refresh token: 7 days, HttpOnly Secure SameSite=Strict cookie; Refresh rotation: old token blacklisted in Redis on use; token family invalidation on theft detection
Password Security	bcrypt with cost factor 12; Argon2id available as upgrade path; pwnedpasswords.com API checked on signup; minimum entropy enforced (zxcvbn score ≥ 3)
Input Validation	Zod schemas on all request bodies + query params; MongoDB injection prevented by Mongoose schema types (no raw query strings); File uploads: type allowlist (JPEG/PNG/WEBP), size limit 5 MB, virus scan stub
Rate Limiting	express-rate-limit: global 100 req/15 min per IP; auth endpoints: 10 req/15 min; payment endpoints: 5 req/min; Redis store for distributed counting
HTTP Security Headers	Helmet.js: CSP, HSTS (1 year, preload), X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy: strict-origin-when-cross-origin
CSRF Protection	Double-submit cookie pattern; CSRF token in custom header validated on state-changing requests; SameSite=Strict on session cookies
CORS	Strict allowlist: web domain, mobile app origin, admin panel; Credentials: true only for allowlisted origins; Preflight cached 600 s
Data Encryption	TLS 1.3 in transit (Nginx + Cloudflare); MongoDB Atlas encryption at rest (AES-256); PII fields (phone, address) encrypted with AES-256-GCM at application layer
Secret Management	All secrets in environment variables; AWS Secrets Manager in production; dotenv-safe enforces all required vars present at startup; No secrets in logs (winston redact plugin)
Dependency Security	Dependabot + npm audit in CI; SBOM generated on every release; Only LTS Node.js versions; base images from official Docker Hub with digest pinning
Audit Logging	Every auth event, role change, payment, refund, and admin action written to AuditLogs collection; Logs are append-only (no update/delete permissions for app user); Shipped to CloudWatch with 90-day retention

13 DEVOPS & CI/CD PIPELINE

Docker, GitHub Actions, monitoring and zero-downtime deployments

CI/CD Pipeline (GitHub Actions)

- on: push to any branch; on: pull_request to main
- Job 1 — lint-and-type-check: ESLint + Prettier check; TypeScript tsc --noEmit (backend); Biome lint (frontend)
- Job 2 — test: Jest unit + integration tests with MongoDB Memory Server; coverage threshold 80%; test report uploaded as artifact
- Job 3 — build: docker buildx build --platform linux/amd64,linux/arm64; multi-stage Dockerfile (builder + production); image tagged with git SHA
- Job 4 — security-scan: npm audit --audit-level=high; Trivy image scan; OWASP Dependency Check
- Job 5 — deploy (main only): Push image to ECR; SSH to EC2 → docker pull + docker compose up -d --no-deps; health-check endpoint polled 10×
- Job 6 — e2e (main only): Cypress tests against staging URL; k6 smoke load test (50 VUs, 30 s); Lighthouse CI report
- Slack notification on failure with commit SHA, failing job, and link to logs

Docker Setup

- Multi-stage Dockerfile: stage 1 (node:20-alpine) installs deps + builds; stage 2 (node:20-alpine) copies dist only — final image ~180 MB
- docker-compose.yml: backend, frontend (Nginx), MongoDB, Redis, BullMQ worker, Prometheus, Grafana — all networked on internal vingo-net bridge
- Named volumes for MongoDB data persistence; bind mount for Nginx config
- Secrets injected via .env file (local) or Docker secrets / AWS SSM (production)
- Health checks on each service; depends_on with condition: service_healthy

Monitoring Stack

Sentry	Error tracking + performance monitoring; release tracking with source maps; alerts on spike or new issue
Prometheus	Custom metrics: order_placed_total, payment_success_total, socket_connections_gauge, api_latency_histogram; scraped every 15 s
Grafana	Dashboards: API latency (p50/p95/p99), DB query times, Redis hit rate, order funnel conversion, queue depth
Winston	Structured JSON logs: level, message, requestId, userId, duration, status; shipped to CloudWatch Logs via winston-cloudwatch
Uptime Robot	External HTTP checks every 1 min; SMS + email on downtime; public status page

14 SCALING & PERFORMANCE STRATEGY

Caching, queues, CDN, horizontal scaling and database optimisation

Redis Caching	Restaurant listings (TTL 10 min); menu pages (TTL 5 min); AI recommendations (TTL 1 h); rate-limit counters; active Socket.IO rooms (pub/sub)
Database Indexing	Compound index (restaurantId, status) on Orders; 2dsphere on Restaurants.location + DeliveryTracking.location; Text index on Restaurants + Menus for full-text search; TTL index on DeliveryTracking (72 h) + Notifications (30 days)
BullMQ Job Queues	Separate queues: email, sms, push, refund, analytics-event, recommendation; Concurrency per queue tuned independently; Failed jobs: 3 retries with exponential backoff; dead-letter queue + alert
Horizontal Scaling	Stateless Node.js app servers behind Nginx upstream; Socket.IO rooms synced via Redis adapter (no sticky sessions needed); AWS Auto Scaling Group: scale out when CPU >60% for 2 min
CDN & Static Assets	React build output served from S3 + CloudFront; Cache-Control: max-age=31536000, immutable for hashed assets; Cloudinary for image transforms (resize, WebP conversion, lazy-load placeholders)
Database Sharding Path	MongoDB Atlas sharding on orderId hash key for Orders collection when single replica set reaches 80% capacity; Read preference: secondaryPreferred for analytics queries
Compression	Nginx gzip on text responses >1 KB; Brotli for modern browsers; HTTP/2 multiplexing via Nginx
Connection Pooling	Mongoose: poolSize 10 per Node.js instance; Redis: ioredis with maxRetriesPerRequest 3; Razorpay SDK: keep-alive agent

15 TESTING STRATEGY

Unit · Integration · E2E · Load tests with coverage gates

Unit Tests (Jest)

- Coverage target: 80% lines, 75% branches; enforced in CI with --coverage --coverageThreshold
- Auth service: password hashing, token generation, refresh rotation, lockout logic
- Order service: state machine transitions, coupon engine, price computation, ETA calculation
- Payment service: Razorpay signature verification, idempotency key logic, refund routing
- Delivery engine: partner scoring algorithm, surge multiplier formula, OTP generation
- Mock strategy: jest.mock() for Razorpay SDK, Nodemailer, Redis client, Cohere API

Integration Tests (Supertest + MongoDB Memory Server)

- Full HTTP request/response cycle against in-memory MongoDB; no external dependencies
- Auth flow: signup → login → refresh → logout → protected route access
- Order flow: create cart → place order → verify payment → accept → deliver
- RBAC: each protected route tested with wrong role → expects 403
- Payment: webhook replay test; duplicate idempotency key rejection test
- Error cases: invalid payloads, missing fields, expired tokens, out-of-stock items

E2E Tests (Cypress)

- Runs against staging environment after every deploy to main branch
- Critical path: user signup → search restaurant → add to cart → checkout → track order
- Owner path: login → accept order → mark ready
- Payment: Razorpay test mode with test card numbers
- Accessibility: axe-core plugin; fails CI on WCAG 2.1 AA violations
- Visual regression: Percy snapshots on key screens; diffs reviewed on PR

Load Tests (k6)

- Smoke test: 10 VUs, 1 min — verifies baseline before every deploy
- Average load: 100 VUs, 10 min — validates P95 <300 ms target
- Stress test: ramp to 500 VUs over 5 min — finds breaking point
- Spike test: 0 → 1000 VUs in 30 s — validates auto-scaling behaviour
- Soak test: 200 VUs, 2 hours — catches memory leaks and connection pool exhaustion
- Thresholds: http_req_duration p(95)<300ms; http_req_failed<1%; checks>99%

16 FOLDER STRUCTURE

Production-grade monorepo with clear domain separation

```
vingo/
■■■ frontend/
■ ■■■ src/
■ ■ ■■■ components/ # Reusable UI (Button, Modal, MapPin…)
■ ■ ■■■ pages/ # Route-level components (Home, Cart, Track…)
■ ■ ■■■ features/ # Redux Toolkit slices (auth, cart, orders…)
■ ■ ■■■ hooks/ # Custom React hooks (useSocket, useLocation…)
■ ■ ■■■ services/ # Axios instances + API call functions
■ ■ ■■■ lib/ # Utilities (formatCurrency, dateHelpers…)
■ ■ ■■■ types/ # TypeScript interfaces and enums
■ ■■■ public/ # Static assets, manifest.json, icons
■ ■■■ vite.config.ts
■
■■■ backend/
■ ■■■ src/
■ ■ ■■■ config/ # DB, Redis, env validation (zod)
■ ■ ■■■ middleware/ # auth, requireRole, errorHandler, rateLimiter
■ ■ ■■■ modules/
■ ■ ■ ■■■ auth/ # controller · service · repository · routes · schema
■ ■ ■ ■■■ restaurants/
■ ■ ■ ■■■ menus/
■ ■ ■ ■■■ orders/
■ ■ ■ ■■■ payments/
■ ■ ■ ■■■ tracking/
■ ■ ■ ■■■ notifications/
■ ■ ■ ■■■ reviews/
■ ■ ■ ■■■ coupons/
■ ■ ■ ■■■ wallet/
■ ■ ■ ■■■ admin/
■ ■ ■■■ workers/ # BullMQ: email, sms, push, refund, recommendations
■ ■ ■■■ socket/ # Socket.IO setup, namespaces, event handlers
■ ■ ■■■ services/ # razorpay, cohere, mapbox, cloudinary wrappers
■ ■ ■■■ utils/ # logger, asyncWrapper, paginate, crypto helpers
■ ■ ■■■ app.ts # Express app factory (no listen here)
■ ■■■ server.ts # HTTP + Socket.IO server bootstrap
■ ■■■ tests/ # __unit__ / __integration__ mirrors module tree
■
■■■ docker-compose.yml
■■■ docker-compose.prod.yml
■■■ .github/workflows/ # ci.yml, deploy.yml, security.yml
■■■ k6/ # smoke.js, load.js, stress.js, soak.js
```

17 SPRINT MILESTONES

6-week execution plan from zero to production deployment

Sprint	Module	Deliverables
Week 1	Auth & RBAC	JWT, refresh tokens, Google OAuth, bcrypt, role-seeding, CSRF, rate-limit middleware
Week 2	Restaurants & Menus	CRUD APIs, image upload to S3/Cloudinary, menu versioning, availability toggles, search + filter
Week 3	Cart, Orders & Payments	Redis cart, Razorpay integration, idempotency keys, webhook processing, order state machine
Week 4	Real-time Tracking	Socket.IO rooms, GPS ingestion, ETA engine, OTP delivery flow, live map WebGL
Week 5	Dashboards & Analytics	Owner revenue charts, Admin heatmaps, churn model, AI recommendation engine (Cohere)
Week 6	Testing & DevOps	Jest unit, Supertest integration, Cypress E2E, k6 load test, Docker, GitHub Actions CI/CD, Sentry, Grafana

Daily Standup Checklist

- All API changes documented in this SRS before merging to main
- New endpoints covered by at least one integration test
- No secrets committed; npm audit run before push
- PR includes screenshot/recording for UI changes; Lighthouse score attached
- BullMQ job failures checked in Grafana dashboard each morning

18 RESUME IMPACT SUMMARY

How to describe Vingo to maximise recruiter and interviewer impact

Vingo is engineered to demonstrate senior-engineer competency across the entire software development lifecycle. The table below maps each project component to the resume bullet and likely interview question it is designed to answer.

Feature	Resume Bullet	Interview Question Answered
Real-time GPS Tracking	Built a real-time order tracking system using Socket.IO with Redis pub/sub adapter for horizontal scaling, processing 10,000 concurrent WebSocket connections	*System design: how do you scale WebSockets?*
Payment Engine	Implemented idempotent payment processing with Razorpay, HMAC-SHA256 webhook verification, and a wallet ledger using MongoDB transactions	*How do you prevent double-charges?*
RBAC Middleware	Designed a 4-role JWT-based access control system with row-level ownership checks enforced at the middleware layer on every protected route	*How do you implement authorisation?*
Caching Strategy	Achieved P95 API latency <300 ms using Redis caching for restaurant listings, menus, and AI recommendation results with tiered TTLs	*How do you improve API performance?*
AI Recommendations	Integrated Cohere Embed API for semantic meal recommendations; stored user taste vectors in pgvector; cached results in Redis	*Have you used AI/ML in a project?*
DevOps Pipeline	Built a full CI/CD pipeline with GitHub Actions: lint → test → Docker build → security scan → deploy → E2E → load test	*How do you ensure code quality?*
Load Testing	Validated system under 500 VU stress test with k6; identified and fixed connection pool exhaustion via soak test	*How do you test performance?*
Fraud Detection	Implemented multi-signal fraud detection: velocity checks, geo-mismatch, COD OTP enforcement, and chargeback rate monitoring	*How do you handle security threats?*
Database Design	Designed 10-collection MongoDB schema with compound indexes, 2dsphere geo-indexes, TTL indexes, and append-only audit trails	*How do you design a database schema?*
Microservice Patterns	Applied domain-module separation, BullMQ async workers, outbox pattern for reliable event publishing, and circuit breaker stubs	*Describe your architecture patterns*

Vingo demonstrates: Distributed Systems · Real-time Architecture · Scalable Caching · Payment Engineering · AI Integration · Security Hardening · CI/CD · Load Testing